

TARAFLOW Referenzfall: Vernetztes Infusionspumpensystem

© Jürgen Messerer · 2026 · Alle Rechte vorbehalten

Zweck dieses Dokuments: Demonstration der TARAflow-Methodik für ein Medizinprodukt. Der Referenzfall zeigt die Stärken des graphbasierten Ansatzes – insbesondere die Integration von Safety und Security sowie die vollständige Rückverfolgbarkeit von der Systemmodellierung bis zur Risikobewertung. Er erfüllt die Anforderungen der MDR und des EU Cyber Resilience Act.

0. Einführung: Warum TARAflow für Medical Devices?

Das Problem klassischer Methoden

Klassische Threat-Modeling-Ansätze analysieren Elemente, nicht Auswirkungen. In der Medizintechnik führt das zu einer gefährlichen Lücke: Ein "Tampering" an einer Konfigurationsdatei ist kein abstraktes Integritätsproblem – bei einer Infusionspumpe ist es eine potenzielle Überdosierung.

Klassisch:
Pumpensteuerung → Tampering (möglich)
→ Denial of Service (möglich)

Kein Patientenbezug, keine Priorisierung, kein klinischer Kontext.

Der TARAflow-Ansatz

TARAflow dreht die Logik um: Assets und ihr klinischer Impact steuern die Analyse.

TARAflow Medical:
DFD-Element → Asset → Klinischer Impact + CIANAAA-Schutzziele → STRIDE → Risk
↑
Was gefährdet den Patienten? (Asset-Kategorie)
Wie schwer ist der Schaden? (Severity nach ISO 14971)
Welches Schutzziel verletzt die Therapie? (C/I/A/N/A/A/A)

Natürliche Prüffragen für Asset-Kategorien

Asset-Typ	Natürliche Prüffrage	Beispiel Infusionspumpe
Data	Was wird gelesen/geschrieben?	Medikamenten-Grenzwerte
Process	Welcher klinische Ablauf muss geschützt werden?	Bolus-Abgabe-Sequenz
System	Was ist das zertifizierte Produkt/Subsystem?	Die Infusionspumpe selbst
Infrastructure	In welcher physischen Umgebung läuft das System?	Klinik-Netzwerk, Dockingstation

Asset-Typ	Natürliche Prüffrage	Beispiel Infusionspumpe
Human	Wer ist am Ende physisch betroffen?	Der Patient

Positionierung

TARAFLOW richtet sich an Hersteller die eine normkonforme TARA mit Nachweispflicht nach IEC 81001-5-1, ISO 14971, MDR Anhang I oder dem EU Cyber Resilience Act durchführen müssen.

1. Systemkontext

Ein vernetztes Infusionspumpensystem besteht aus der physischen Pumpe mit eingebetteter Firmware, einer Dockingstation zur Netzwerkanbindung, einem zentralen Medikamentendatenbank-Service (Drug Library) sowie einem Krankenhausinformationssystem (HIS/EMR). Das System hat direkte Patientenrelevanz, da fehlerhafte Medikamentengaben zu schwerer Verletzung oder Tod führen können.

Systemgrenze: Das Produkt/System unter Analyse umfasst Pumpen-Firmware, Drug-Library-Service, Kommunikationsmodul und alle Schnittstellen nach aussen.

Regulatorischer Kontext: MDR (EU) 2017/745, ISO 14971 (Risikomanagement), IEC 81001-5-1 (Security for Health Software), EU Cyber Resilience Act.

2. DFD-Modellierung

2.1 Elemente

External Entities:

- **Patient** – Empfänger der Therapie (primäres Schutzobjekt)
- **Pflegepersonal** – Bedienung vor Ort, Medikationsstart, Parametereinstellung
- **Klinik-IT (HIS/EMR)** – Zentrales System für Patientendaten und Drug-Library-Updates
- **Wartungstechniker** – Kalibrierung, Firmware-Updates, technischer Service

Processes:

- **Pumpensteuerung** – Präzisionsregelung des Motors, Flussratenausführung
- **Drug-Library-Service** – Validierung von Parametern gegen Sicherheitsgrenzwerte (DERS)
- **Alarm-Manager** – Überwachung auf Okklusion, Luft, Akku-Zustand, Grenzwert-Verletzung
- **Kommunikationsmodul** – Datenaustausch via Klinik-WLAN

Data Stores:

- **Drug Library (lokal)** – Lokale Kopie der Medikamenten-Grenzwerte (DERS)
- **Therapie-Profil** – Aktuell eingestellte Parameter für den Patienten
- **Audit-Log** – Revisionssichere Aufzeichnung aller Bedienvorgänge
- **Firmware-Speicher** – Gespeicherte Firmware-Images und Konfiguration

Data Flows (nach TARAFLOW Naming-Konvention):

- → **send BolusCommand [cmd]** – Pflegepersonal/HIS → Pumpensteuerung
- → **update DrugLibrary [req]** – Klinik-IT → Drug Library (lokal)
- → **recv DrugLibrary [resp]** – Drug Library → Drug-Library-Service

- → `stream VitalData [stream]` – Pumpensteuerung → Monitoring-Zentrale
- → `send Alarm [event]` – Alarm-Manager → Schwesternruf
- → `send FirmwareUpdate [req]` – Wartungstechniker → Firmware-Speicher
- → `send DiagData [resp]` – Pumpensteuerung → Wartungstechniker

Trust Boundaries:

- – **Klinik-Netzwerk-Grenze** – Externe Netzwerkgrenze (WLAN/Internet)
- – **Gerät-Grenze** – Zwischen Klinik-IT und Pumpen-intern
- – **Safety-Grenze** – Zwischen Drug-Library-Service und Pumpensteuerung
- – **Physischer Zugang** – Wartungsschnittstellen (USB, Service-Port)

3. Asset-Modellierung und Beziehungen

TARAflow verwendet fünf Asset-Kategorien die gemeinsam alle schützenswerten Güter eines Systems abdecken. Die Beziehungen zwischen DFD-Elementen und Assets sind typisiert. Sie beschreiben präzise wie ein Element ein Asset berührt. Das ist die Grundlage für die assetbasierte STRIDE-Analyse.

Safety Annotation Layer: Safety-Aspekte werden als optionaler Annotation Layer auf bestehenden Beziehungen modelliert. Wird an einer Beziehung eine SafetyAnnotation gesetzt (`relevance: 'direct', impact: 'fatality'`), signalisiert der Analyst dass dieses Asset über diese Beziehung direkt safety-relevant ist. Der Safety Impact am Asset kann abgeleitet (`derived`) oder manuell (`manual`) gesetzt werden – bei manueller Setzung ist eine Begründung (`rationale`) Pflicht.

3.1 Data Assets

Identifizierte Data Assets: Medikamenten-Grenzwerte, Flussrate, Therapie-Parameter, Alarm-Schwellen, Firmware-Image, Audit-Daten, Patienten-ID.

Beziehungen:

```
DataStore "Drug Library (lokal)"
└─ stores → Data Asset "Medikamenten-Grenzwerte"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                rationale: 'Falsche Grenzwerte → toxische Überdosierung
möglich' }

DataStore "Therapie-Profil"
├─ stores → Data Asset "Flussrate"
│   └─ safety: { relevance: 'direct', impact: 'fatality',
                rationale: 'Falsche Flussrate → Über-/Unterdosierung' }
└─ stores → Data Asset "Therapie-Parameter"
    └─ safety: { relevance: 'direct', impact: 'irreversible_injury' }

DataStore "Firmware-Speicher"
└─ stores → Data Asset "Firmware-Image"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                rationale: 'Kompromittierte Firmware → vollständige
Systemkontrolle' }

DataStore "Audit-Log"
└─ stores → Data Asset "Audit-Daten"
```

```

Process "Pumpensteuerung"
├─ reads    → Data Asset "Flussrate"
└─ reads    → Data Asset "Therapie-Parameter"

Process "Drug-Library-Service"
└─ reads    → Data Asset "Medikamenten-Grenzwerte"

Process "Alarm-Manager"
├─ reads    → Data Asset "Alarm-Schwellen"
└─ creates  → Data Asset "Audit-Daten"

DF "send BolusCommand [cmd]"
└─ transports → Data Asset "Flussrate"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                 rationale: 'Manipulation ermöglicht lebensgefährliche
Bolusgabe' }

DF "update DrugLibrary [req]"
└─ transports → Data Asset "Medikamenten-Grenzwerte"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                 rationale: 'Kompromittiertes Update → falsche
Sicherheitsgrenzwerte' }

DF "send FirmwareUpdate [req]"
└─ transports → Data Asset "Firmware-Image"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                 rationale: 'Manipuliertes Firmware-Update → vollständige
Gerätekontrolle' }

```

3.2 Process Assets

Process Assets sind **klinische Abläufe** die als solche schützenswert sind – unabhängig davon welche Software sie implementiert. Ein Angreifer muss nicht Daten manipulieren; er kann den Ablauf selbst kompromittieren (Bypass, Unterbrechung, Reihenfolgeänderung).

Identifizierte Process Assets: Bolus-Abgabe-Sequenz, Medikamenten-Validierungsablauf, Alarm-Eskalationsablauf, Wartungs- und Kalibrierungsablauf.

Beziehungen:

```

Process "Pumpensteuerung"
└─ is_an → Process Asset "Bolus-Abgabe-Sequenz"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                 physicalHazardPotential: 'high',
                 rationale: 'Unkontrollierte Ausführung →
Über-/Unterdosierung' }

Process "Drug-Library-Service"
└─ is_an → Process Asset "Medikamenten-Validierungsablauf"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                 rationale: 'Bypass der Validierung → Grenzwerte nicht
eingehalten' }

```

```

Process "Alarm-Manager"
└ is_an → Process Asset "Alarm-Eskalationsablauf"
  └ safety: { relevance: 'direct', impact: 'irreversible_injury',
              rationale: 'Ausfall → keine Warnung bei kritischem
Patientenzustand' }

EE "Pflegepersonal"
└ invokes → Process Asset "Bolus-Abgabe-Sequenz"

EE "Wartungstechniker"
└ invokes → Process Asset "Wartungs- und Kalibrierungsablauf"
└ monitors → Process Asset "Bolus-Abgabe-Sequenz"

```

3.3 System Assets

System Assets sind das **zertifizierte Produkt und seine funktionalen Subsysteme** – was auf dem Typenschild steht und was als MDR-Einheit reguliert wird.

Identifizierte System Assets: Infusionspumpe, Drug-Library-Server, Monitoring-System.

Beziehungen:

```

Process "Pumpensteuerung"
└ is_an → System Asset "Infusionspumpe"
  └ safety: { relevance: 'direct', impact: 'fatality',
              physicalHazardPotential: 'high' }

Process "Drug-Library-Service"
└ depends_on → System Asset "Drug-Library-Server"

Process "Alarm-Manager"
└ uses [network] → System Asset "Monitoring-System"

Process "Kommunikationsmodul"
└ is_an → System Asset "Infusionspumpe"

EE "Wartungstechniker"
└ uses [local] → System Asset "Infusionspumpe"

EE "Klinik-IT (HIS/EMR)"
└ uses [network] → System Asset "Drug-Library-Server"

```

3.4 Infrastructure Assets

Infrastructure Assets beschreiben die **physische und netzwerktechnische Umgebung** in der das System betrieben wird – Gebäude, Räume, Verkabelung, Stromversorgung, Netzwerk-Infrastruktur und physische Zugangspunkte.

Identifizierte Infrastructure Assets: Klinik-Netzwerk-Infrastruktur, Dockingstation, Patientenzimmer (physischer Zugang), Stromversorgung/USV.

Beziehungen:

```

Process "Kommunikationsmodul"
└─ uses [network] → Infrastructure Asset "Klinik-Netzwerk-Infrastruktur"
    └─ safety: { relevance: 'indirect',
                rationale: 'Netzwerkausfall → Drug-Library-Updates nicht
möglich' }

Process "Pumpensteuerung"
└─ depends_on → Infrastructure Asset "Stromversorgung/USV"
    └─ safety: { relevance: 'indirect',
                rationale: 'Stromausfall → Therapieunterbrechung' }

EE "Wartungstechniker"
└─ accesses [local] → Infrastructure Asset "Dockingstation"
└─ accesses [internal] → Infrastructure Asset "Patientenzimmer (physischer
Zugang)"

EE "Pflegepersonal"
└─ accesses [local] → Infrastructure Asset "Patientenzimmer (physischer
Zugang)"

Process "Pumpensteuerung"
└─ is_an → Infrastructure Asset "Dockingstation"
    └─ rationale: 'Physische Verbindung zur Netzwerkanbindung'

```

3.5 Human Assets

Human Assets beschreiben Menschen als Schutzobjekte – sowohl Safety (physische Unversehrtheit) als auch Privacy (Datenschutz nach MDR/DSGVO).

Identifizierte Human Assets: Patient, Pflegepersonal, Wartungstechniker.

Beziehungen:

```

EE "Patient"
└─ is_an → Human Asset "Patient"
    └─ Properties: { isProtectionTarget: true, safetyImpact: 'fatality',
                    rationale: 'Direkte Abhängigkeit von korrekter
Medikamentenzufuhr' }

Process "Pumpensteuerung"
└─ affects_safety → Human Asset "Patient"
    └─ safety: { relevance: 'direct', impact: 'fatality',
                rationale: 'Unkontrollierte Flussrate → Über-/Unterdosierung'
}

Process "Alarm-Manager"
└─ affects_safety → Human Asset "Patient"
    └─ safety: { relevance: 'direct', impact: 'irreversible_injury',
                rationale: 'Ausfall Alarmierung → verzögerte Intervention' }

Process "Drug-Library-Service"
└─ affects_safety → Human Asset "Patient"
    └─ safety: { relevance: 'direct', impact: 'fatality',

```

```

    rationale: 'Falsche Validierung → Grenzwerte nicht
eingehalten' }

EE "Wartungstechniker"
└ exposes → Human Asset "Patient"
    └ rationale: 'Kompromittierter Service-Zugang ermöglicht Manipulation der
Pumpe'

Process "Kommunikationsmodul"
└ affects_privacy → Human Asset "Patient"
    └ rationale: 'Patientendaten und Therapieparameter über Klinik-Netz
übertragen'

```

4. Asset Impact- und Schutzziel-Bewertung

Erst nachdem die Beziehungen im Graphen festgelegt sind, wird bewertet was jedes Asset schützenswert macht und warum. Die Safety-Relevanz ergibt sich direkt aus den Beziehungen.

4.0 Formale Aggregationslogik

Die aggregierte Asset-Kritikalität ergibt sich aus zwei unabhängigen Domänen:

Domäne	Kriterien
Business / Organisatorisch	Financial Damage, Regulatory/Compliance (MDR/FDA), Reputation, Operational Impact, Affected Users, Recoverability
Physical / Safety	Safety (Patientengefährdung), Physical Asset Damage, Environmental Impact

Aggregationsmatrix:

Business Impact	Safety Impact	relevance	Aggregiert
beliebig	HIGH (fatality / irreversible_injury)	direct	CRITICAL
beliebig	HIGH (fatality / irreversible_injury)	indirect	HIGH+
CRITICAL	– oder MED	–	CRITICAL
HIGH	MED (indirect, Hop 1)	–	HIGH
HIGH	–	–	HIGH
MEDIUM	MED (indirect, Hop 1)	–	MEDIUM+
MEDIUM	–	–	MEDIUM
LOW	MED (indirect, Hop 1)	–	MEDIUM
LOW	–	–	LOW

Hinweis zu HIGH+/MEDIUM+: Diese Zwischenwerte bedeuten "oberes Ende der Stufe" und können durch hohe Likelihood in der Risk-Tabelle zu CRITICAL/HIGH eskalieren. Die finale Risikoeinstufung ergibt sich aus Impact × Likelihood – nicht aus dem aggregierten Kritikalitätswert allein.

Safety Override Rule (ISO 14971 / MDR):

```
IF safetyImpact ∈ { 'fatality', 'irreversible_injury' }  
  AND relevance === 'direct'  
THEN aggregatedCriticality := CRITICAL  
  strideDepth              := 'vertieft'  
  riskPriority              := 1  
  
IF safetyImpact ∈ { 'fatality', 'irreversible_injury' }  
  AND relevance === 'indirect'  
THEN aggregatedCriticality := MAX(businessImpact, HIGH+)  
  strideDepth              := 'fokussiert'  
  riskPriority              := 2
```

Begründung:

Severity ist eine Eigenschaft des Schadens – nicht des Assets.
Ein Asset das fatality nur systemisch beeinflusst (indirect) hat keinen unmittelbaren Steuerungseinfluss. Die Override-Regel greift nur dort wo das Asset die physische Aktion direkt kontrolliert.

4.0a Safety Impact als abgeleitetes Feld (Derived/Manual Pattern)

SafetyAnnotation auf Beziehung	→ physicalImpact am Asset
impact: 'fatality'	→ HIGH (derived)
impact: 'irreversible_injury'	→ HIGH (derived)
relevance: 'indirect' (transitiv, Hop 1)	→ MED (derived)
keine Annotation	→ LOW (derived, default)

- **physicalImpactSource: "derived"** → keine zusätzliche Dokumentationspflicht
- **physicalImpactSource: "manual"** → Rationale wird verbatim im Audit-Report dokumentiert

4.1 Schutzziel-Ableitung aus Beziehungstyp (Auszug Medical)

Beziehungstyp	Primäre Schutzziele	Medical-Kontext
stores	Integrity, Availability	Drug Library: Verfügbarkeit bei Therapiestart
transports	Integrity, Confidentiality, Authentication	Bolus-Befehle: Herkunft verifizierbar
affects_safety	Availability, Integrity	Pumpensteuerung: darf nicht ausfallen
is_an (Process)	Integrity, Availability	Bolus-Abgabe-Sequenz: korrekte Ausführung
accesses [local]	Authorization, Non-Repudiation	Dockingstation: wer hat zugegriffen?
affects_privacy	Confidentiality, Accountability	Patientendaten: MDR/DSGVO

4.2 Data Assets

Lesehilfe Safety Impact (physicalImpact): Primär aus der SafetyAnnotation im DFD-Tab abgeleitet (*derived*). HIGH = *fatality* oder *irreversible_injury*, MED = *indirect* (Hop 1), – = keine Annotation.

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Medikamenten-Grenzwerte	Integrity, Availability, Authorization	HIGH (Regulatory)	HIGH (fatality)	CRITICAL
Flussrate	Integrity, Availability	HIGH (Operational)	HIGH (fatality)	CRITICAL
Therapie-Parameter	Integrity, Availability, Authorization	HIGH (Operational)	HIGH (irreversible_injury)	CRITICAL
Firmware-Image	Integrity, Availability, Authentication	HIGH (Operational)	HIGH (fatality)	CRITICAL
Alarm-Schwellen	Integrity, Availability	HIGH (Operational)	HIGH (irreversible_injury)	CRITICAL
Patienten-ID	Confidentiality, Integrity, Accountability	MEDIUM (Privacy/MDR)	MED (indirect)	HIGH
Audit-Daten	Integrity, Non-Repudiation, Accountability	MEDIUM (Regulatory)	–	MEDIUM

4.3 Process Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Bolus-Abgabe-Sequenz	Integrity, Availability, Authorization	HIGH (Operational)	HIGH (fatality)	CRITICAL
Medikamenten-Validierungsablauf	Integrity, Availability	HIGH (Regulatory)	HIGH (fatality)	CRITICAL
Alarm-Eskalationsablauf	Availability, Integrity	HIGH (Operational)	HIGH (irreversible_injury)	CRITICAL
Wartungs- und Kalibrierungsablauf	Integrity, Authorization, Non-Repudiation	MEDIUM (Operational)	MED (indirect)	HIGH

4.4 System Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Infusionspumpe	Availability, Integrity, Authorization	HIGH (Operational)	HIGH (fatality)	CRITICAL
Drug-Library-Server	Availability, Integrity	HIGH (Regulatory)	HIGH (fatality)	CRITICAL
Monitoring-System	Availability	HIGH (Operational)	MED (indirect)	CRITICAL

4.5 Infrastructure Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Aggregiert
Klinik-Netzwerk-Infrastruktur	Availability, Authentication	HIGH (Operational)	MED (indirect)	CRITICAL
Dockingstation	Availability, Authorization	MEDIUM (Operational)	MED (indirect)	HIGH
Stromversorgung/USV	Availability	HIGH (Operational)	MED (indirect)	CRITICAL
Patientenzimmer (physischer Zugang)	Authorization, Non-Repudiation	MEDIUM (Operational)	MED (indirect)	HIGH

4.6 Human Assets

Asset	Schutzziele (CIANAAA)	Business Impact	Safety Impact	Protection Target	Aggregiert
Patient	Availability (Safety), Confidentiality	–	HIGH (fatality)	✓	CRITICAL
Pflegepersonal	Availability (Safety), Confidentiality	MEDIUM (Privacy)	HIGH (irreversible_injury)	✓	CRITICAL
Wartungstechniker	Confidentiality, Authorization	MEDIUM (Privacy)	MED (indirect)	–	HIGH

4.7 Ergebnis der Bewertung

Von 18 identifizierten Assets sind 12 als CRITICAL eingestuft – ausnahmslos wegen Safety Impact. Der Patient als Protection Target ist der Endpunkt aller Safety-Ketten. Seine Kritikalität ergibt sich aus den **affects_safety**- und **exposes**-Beziehungen im Graphen – begründet und rückverfolgbar.

5. Erkenntnissprung: Was TARAflow im Medical-Bereich sichtbar macht

5.1 Beispiel: Drug-Library-Update als Safety-Angriffsvektor

Klassisch:

```
DF "update DrugLibrary [req]" → Tampering (möglich)
```

Kein Patientenbezug, keine Priorisierung.

TARAFLOW:

```
DF "update DrugLibrary [req]"
└ transports → Data Asset "Medikamenten-Grenzwerte" [Integrity: CRITICAL,
fatality]
  └ reads: □ Drug-Library-Service
    └ is_an → Process Asset "Medikamenten-Validierungsablauf" [fatality]
      └ affects_safety → Human Asset "Patient" [Protection Target]
```

- Threat: Manipulation des Drug-Library-Updates
- Konsequenz: Falsche Sicherheitsgrenzwerte → toxische Überdosierung
- Priorität: CRITICAL (Safety Override Rule)
- Massnahme: Digitale Signatur + Integritätsprüfung vor Import

5.2 Beispiel: Wartungszugang als versteckter Safety-Angriffsvektor

Klassisch:

```
EE "Wartungstechniker" → Spoofing (möglich), Tampering (möglich)
```

TARAFLOW:

```
EE "Wartungstechniker"
└ accesses [local] → Infrastructure Asset "Dockingstation"
  └ uses [local] → System Asset "Infusionspumpe" [CRITICAL]
    └ invokes → Process Asset "Wartungs- und Kalibrierungsablauf"
      └ transports → Data Asset "Firmware-Image" [fatality]
        └ exposes → Human Asset "Patient" [fatality]
```

- Was sichtbar wird:
 - Der Wartungszugang ist nicht nur ein IT-Risiko –
 - physischer Zugang zur Dockingstation ist ein direkter Safety-Angriffsvektor.
 - Kompromittierter Techniker → Firmware-Manipulation
 - Pumpe verhält sich unkontrolliert → Patientengefährdung

5.3 Beispiel: Alarm-Eskalationsablauf als Single Point of Safety-Failure

Klassisch: Alarm-Manager als normaler Prozess – STRIDE wie alle anderen.

TARAFLOW:

Process "Alarm-Manager"

└ is_an → Process Asset "Alarm-Eskalationsablauf" [Availability: CRITICAL, irreversible_injury]

└ affects_safety → Human Asset "Patient"

→ Was sichtbar wird:

Der Alarm-Eskalationsablauf ist der einzige Ablauf der bei kritischen Patientenzuständen eine Intervention auslöst.

Denial of Service auf Alarm-Manager = kein Alarm möglich

= Pflegepersonal wird nicht benachrichtigt = kein Patientenschutz.

6. Asset-Priorisierung

6.1 Kritikalitätsbewertung Medical-Assets (Auszug)

Asset	Business Impact	Safety Impact	Aggregiert	STRIDE-Tiefe
Medikamenten-Grenzwerte	HIGH (Regulatory)	HIGH (Fatality)	CRITICAL	Vertieft
Bolus-Abgabe-Sequenz	HIGH (Operational)	HIGH (Fatality)	CRITICAL	Vertieft
Medikamenten-Validierungsablauf	HIGH (Regulatory)	HIGH (Fatality)	CRITICAL	Vertieft
Infusionspumpe	HIGH (Operational)	HIGH (Fatality)	CRITICAL	Vertieft
Stromversorgung/USV	HIGH (Operational)	MED (indirect)	CRITICAL	Fokussiert
Klinik-Netzwerk-Infrastruktur	HIGH (Operational)	MED (indirect)	CRITICAL	Fokussiert
Wartungs- und Kalibrierungsablauf	MEDIUM	MED (indirect)	HIGH	Fokussiert
Audit-Daten	MEDIUM (Regulatory)	–	MEDIUM	Hochstufig

6.2 Safety Override Rule

Assets mit **safetyImpact: 'fatality'** oder **safetyImpact: 'irreversible_injury'** erhalten automatisch die höchste Analysepriorität, unabhängig vom Business Impact (ISO 14971).

7. STRIDE-Analyse (Auszug)

Threat T-01: Manipulation der Infusionsrate (Bolos-Angriff)

Feld	Inhalt
DFD-Element	DF <code>send BolusCommand [cmd]</code>
Asset-Beziehung	<code>transports</code> → Data Asset <code>"Flussrate"</code>
STRIDE-Kategorie	Tampering / Spoofing
CIANAAA-Verletzung	Integrity, Authentication
Schutzziel	Bolos-Befehle nur von autorisiertem Pflegepersonal
Angriffspfad	Kompromittierung Zentralstation → unautorisierter Bolus-Befehl → Ausführung ohne Validierungsablauf
Safety-Konsequenz	<code>affects_safety</code> → Human Asset <code>"Patient"</code> → fatality
Priorität	CRITICAL (Safety Override Rule)
STRIDE-Tiefe	Vertieft (Trust Boundary + CRITICAL Asset + Safety)

Threat T-02: Manipulation des Drug-Library-Updates

Feld	Inhalt
DFD-Element	DF <code>update DrugLibrary [req]</code>
Asset-Beziehung	<code>transports</code> → Data Asset <code>"Medikamenten-Grenzwerte"</code>
STRIDE-Kategorie	Tampering
CIANAAA-Verletzung	Integrity, Authentication
Schutzziel	Drug-Library-Updates nur von verifizierter Klinik-IT mit Integritätsnachweis
Angriffspfad	MITM im Klinik-Netz → manipuliertes Update → DERS-Grenzwerte erhöht → toxische Dosen nicht blockiert
Safety-Konsequenz	<code>Medikamenten-Grenzwerte</code> → <code>Medikamenten-Validierungsablauf</code> → <code>Patient</code> → fatality
Priorität	CRITICAL (Safety Override Rule)
STRIDE-Tiefe	Vertieft (Trust Boundary + CRITICAL Asset + Safety)

Threat T-03: Denial of Service Alarm-Eskalationsablauf

Feld	Inhalt
DFD-Element	Process <code>Alarm-Manager</code>
Asset-Beziehung	<code>is_an</code> → Process Asset <code>"Alarm-Eskalationsablauf"</code>
STRIDE-Kategorie	Denial of Service

Feld	Inhalt
CIANAAA-Verletzung	Availability
Schutzziel	Alarm-Eskalation muss jederzeit verfügbar sein
Angriffspfad	Netzwerk-Flooding → Alarm-Manager nicht erreichbar → kein Alarm bei Okklusion → verzögerte Intervention
Safety-Konsequenz	affects_safety → Patient → irreversible_injury
Priorität	CRITICAL (Safety Override Rule)
STRIDE-Tiefe	Vertieft (CRITICAL Asset + Safety)

Threat T-04: Kompromittierter Wartungszugang via Dockingstation

Feld	Inhalt
DFD-Element	EE <i>Wartungstechniker</i> via <i>accesses [local]</i> → <i>Dockingstation</i>
Asset-Beziehung	<i>transports</i> → <i>Data Asset "Firmware-Image"</i>
STRIDE-Kategorie	Tampering / Elevation of Privilege
CIANAAA-Verletzung	Integrity, Authorization, Non-Repudiation
Schutzziel	Firmware-Updates nur signiert und von autorisiertem Personal
Angriffspfad	Physischer Zugang zur Dockingstation → unsigniertes Firmware-Image → vollständige Gerätekontrolle
Safety-Konsequenz	<i>indirect via System Asset "Infusionspumpe"</i> → fatality
Priorität	CRITICAL (Safety Override Rule)
STRIDE-Tiefe	Vertieft (CRITICAL Asset + physischer Zugang)

8. Risk-Tabelle (Auszug)

ID	Threat	Likelihood	Impact	Risk	Safety Override	Massnahme	Prio
T-01	Bolus-Manipulation	MEDIUM	CRITICAL	HIGH	✅ fatality	mTLS + Bestätigung kritischer Befehle am Gerät	1
T-02	Drug-Library-Manipulation	MEDIUM	CRITICAL	HIGH	✅ fatality	Digitale Signatur + Integritätsprüfung vor Import	1

ID	Threat	Likelihood	Impact	Risk	Safety Override	Massnahme	Prio
T-03	DoS Alarm-Eskalation	HIGH	CRITICAL	CRITICAL	✓ irreversible_injury	Offline-Fallback + dedizierter Alarm-Kanal	1
T-04	Firmware via Dockingstation	LOW	CRITICAL	MEDIUM	✓ fatality	Secure Boot + signierte Firmware + physische Zugangskontrolle	1
T-05	Patienten-ID Disclosure	MEDIUM	HIGH	HIGH	–	TLS + Authentifizierung HIS-Kommunikation	2

9. Der TARAflow-Graph als Single Source of Truth

Knoten: DFD-Elemente + Assets (5 Kategorien)
Kanten: typisierte Beziehungen mit optionaler SafetyAnnotation

9.1 Die vollständige Safety-Kette im Graphen

```
Angriff auf Drug-Library-Update (Cyber-Bedrohung)
  ↓ transports [Integrity verletzt]
Manipulation Medikamenten-Grenzwerte (Data Asset)
  ↓ reads
Fehlerhafte Ausführung Medikamenten-Validierungsablauf (Process Asset)
  ↓ is_an → Bolus-Abgabe-Sequenz wird unkontrolliert ausgeführt
Falsche Flussrate erreicht Patienten (Data Asset)
  ↓ affects_safety
Überdosierung des Patienten (Human Asset – Protection Target) ●
```

Jeder Schritt ist im Graph explizit modelliert. Daraus folgt automatisch die höchste Threat-Priorität – begründet, auditierbar, normkonform nach ISO 14971 und MDR.

10. Positionierung für die MDR-Zulassung

Anforderung	MDR / ISO 14971	TARAflow-Nachweis
Risikomanagement-Akte	MDR Anhang I, §3	TARA vollständig rückverfolgbar
Security-Risk-Assessment	IEC 81001-5-1	Graph-basierte STRIDE-Analyse
Traceability Massnahmen	MDR Art. 10	Massnahme → Threat → Asset → DFD-Element
Safety/Security-Integration	ISO 14971 §4	Safety Annotation Layer im Security-Modell

11. Nächste Schritte mit diesem Referenzfall

1. **Graph vervollständigen** – alle Asset-Beziehungen formal in TARAflow erfassen
 2. **STRIDE vollständig** – alle Threats für CRITICAL Assets systematisch ableiten
 3. **Attack Trees** – für T-01 und T-02 als Beispiel für Phase 3 TARAflow
 4. **Massnahmen vollständig** – alle Threats mit Normnachweis (IEC 81001-5-1, MDR)
 5. **Export** – alle Diagramme und Dokumente automatisch aus dem Graph generieren
-